Лабораторийн ажил №4

Сэдэв: Process State, Process Priority, Context Switching-ийн симуляци

Гүйцэтгэсэн: Н.Мөнхпүрэв, 23B1NUM0114

1. ОРШИЛ

Энэ лабораторийн ажилд үйлдлийн системийн процесс удирдлагын үндсэн ойлголтууд болох PCB, процессын төлөв, priority scheduling болон context switching-ийг C хэл дээр энгийн simulation байдлаар хийв. Программ бодит kernel scheduler биш боловч процессуудыг удирдлагын мэдээлэлтэй объект гэж авч үзэх санааг харуулж байна.

Програмд гурван процесс үүсгэсэн. Process 1 нь random тоо эрэмбэлэх, Process 2 нь матриц нэмэх, Process 3 нь тэмдэгт мөрийн давтамж тоолох ажил хийдэг. Scheduler нь READY төлөвтэй процессуудаас priority хамгийн өндөр процессыг сонгож ажиллуулна.

2. ЗОРИЛГО

PCB бүтэц ашиглан процесс бүрийн мэдээллийг хадгалж, READY, RUNNING, WAITING, TERMINATED төлөвүүдийн шилжилтийг харуулах priority scheduler simulation хийх.

1. struct PCB бүтцийг тодорхойлж процессын мэдээллийг хадгалах.
2. 3 процесс үүсгэж, тус бүр өөр логиктой ажиллуулах.
3. READY төлөвтэй процессуудаас priority-гаар дараагийн процессыг сонгох.
4. Context switching явагдаж байгааг console дээр хэвлэх.
5. WAITING болон TERMINATED төлөвийн шилжилтийг simulation-д оруулах.
6. gcc болон Makefile ашиглан хөрвүүлж шалгах.

3. ОНОЛЫН СУДАЛГАА

3.1 PCB ба процессын төлөв

Процесс гэдэг нь санах ойд ачаалагдсан, ажиллаж байгаа програм юм. Үйлдлийн систем процесс бүрийг зөвхөн код гэж үзэхгүй, харин process id, төлөв, priority, counter зэрэг удирдлагын мэдээлэлтэй объект гэж үздэг. Энэ мэдээллийг PCB буюу Process Control Block дотор хадгална.

Процессын төлөв нь тухайн процесс яг одоо ямар байдалтай байгааг илэрхийлнэ. READY нь CPU авахад бэлэн, RUNNING нь яг одоо ажиллаж байгаа, WAITING нь ямар нэг хугацаа эсвэл I/O хүлээж байгаа, TERMINATED нь ажлаа дуусгасан гэсэн утгатай.

3.2 Priority scheduling ба context switching

Priority scheduling нь READY төлөвтэй процессууд дундаас хамгийн өндөр priority-той процессыг сонгож CPU дээр ажиллуулдаг арга юм. Энэ лабораторид priority-ийн тоо бага байх тусам илүү өндөр эрхтэй гэж үзсэн. Иймээс Priority 1 нь Priority 2-оос түрүүлж ажиллана.

Context switching гэдэг нь CPU нэг процессоос нөгөө процесс руу шилжих үйлдэл юм. Бодит үйлдлийн системд register, program counter зэрэг context хадгалагдаж сэргээгддэг. Энэ simulation-д бодит register хадгалахгүй, харин төлөв солигдож байгааг текстээр харуулсан.

4. ХЭРЭГЖҮҮЛЭЛТ

Програмыг нэг үндсэн эх файл болох Lab4.c болон жижиг Makefile ашиглан хийсэн. Даалгаврын шаардлагын дагуу кодыг функцүүдэд хувааж бичсэн.

Файл	Үүрэг	Тайлбар
Lab4.c	Үндсэн программ	PCB, scheduler, процессын логик, context switch бүгд энд байна.
Makefile	Хөрвүүлэх файл	make командаар gcc ашиглан lab4 executable үүсгэнэ.
Readme.md	Товч тайлбар	Ажиллуулах команд болон тайлангийн асуултын богино хариулт.

4.1 PCB бүтэц

PCB бүтэц нь процессын удирдлагад хэрэгтэй мэдээллийг хадгална.

```
struct PCB {
    int process_id;
    int priority;
    int state;
    int process_counter;
    int burst_time;
    int waiting_time;
};
```

4.2 Процессуудын логик

Процесс	Priority	Ажил	Төлөвийн онцлог
Process 1	1	5 random тоо үүсгээд өсөхөөр эрэмбэлнэ.	Burst дуусах хүртэл READY болж буцна.
Process 2	2	2x2 матриц нэмнэ.	Эхний ажилласны дараа 2 алхам WAITING болно.
Process 3	3	operating system string-ийн үсгийн давтамж тоолно.	Burst дуусахад TERMINATED болно.

4.3 Scheduler-ийн шийдэл

selectNextProcess() функц READY төлөвтэй процессуудыг шалгаж, priority хамгийн бага утгатай процессыг сонгодог. Хэрэв READY процесс байхгүй бол CPU idle гэж хэвлэнэ.

```
if (processes[i].state == READY) {
    if (selected == -1 || processes[i].priority < processes[selected].priority) {
        selected = i;
    }
}
```

4.4 Context switching

contextSwitch() функц CPU аль процессоос аль процесс руу шилжиж байгааг хэвлэнэ. Хэрэв эхний процесс ажиллаж байгаа бол CPU -> Process X гэж гарна.

```
[Context Switch] CPU -> Process 1
[Context Switch] Process 1 -> Process 2
```

5. ТУРШИЛТ БА ҮР ДҮН

Програмыг Linux орчинд gcc ашиглан хөрвүүлж ажиллуулсан.

```
make
./lab4
make clean
```

Шалгалт	Үр дүн	Тайлбар
Compile	Амжилттай	gcc -Wall -Wextra -std=c99 алдаагүй хөрвүүлсэн.
Scheduler	Амжилттай	READY процессуудаас priority-гаар сонгож ажилласан.
WAITING төлөв	Амжилттай	Process 2 хоёр алхам WAITING болсон.
Context switch	Амжилттай	CPU болон процесс хоорондын шилжилт хэвлэгдсэн.
Termination	Амжилттай	Бүх процесс TERMINATED болсон.

Жишээ гаралтын хэсэг:

```
Lab 4: Process State, Priority and Context Switching Simulation
CPU Scheduler selected Process 1
[Context Switch] CPU -> Process 1
CPU Scheduler selected Process 1
CPU Scheduler selected Process 1
Process 1 finished. State changed to TERMINATED
PID: 1 | Priority: 1 | State: TERMINATED | Counter: 3 | Burst: 0 | Waiting: 0
CPU Scheduler selected Process 2
[Context Switch] Process 1 -> Process 2
Process 2 needs I/O. State changed to WAITING
PID: 1 | Priority: 1 | State: TERMINATED | Counter: 3 | Burst: 0 | Waiting: 0
PID: 2 | Priority: 2 | State: WAITING    | Counter: 1 | Burst: 2 | Waiting: 2
Process 2 is WAITING. Remaining wait = 1
```

```
CPU Scheduler selected Process 3
[Context Switch] Process 2 -> Process 3
PID: 1 | Priority: 1 | State: TERMINATED | Counter: 3 | Burst: 0 | Waiting: 0
PID: 2 | Priority: 2 | State: WAITING    | Counter: 1 | Burst: 2 | Waiting: 1
Process 2 is WAITING. Remaining wait = 0
```

6. ТАЙЛАНГИЙН АСУУЛТУУД

1. PCB гэж юу вэ?

PCB буюу Process Control Block нь процессын удирдлагын мэдээллийг хадгалдаг бүтэц юм. Үүнд процессын дугаар, төлөв, priority, counter зэрэг мэдээлэл орно.

2. Process state ямар үүрэгтэй вэ?

Process state нь тухайн процесс одоо ямар байдалтай байгааг илэрхийлнэ. Үйлдлийн систем scheduler хийхдээ энэ мэдээллийг ашигладаг.

3. READY ба RUNNING төлөвийн ялгаа юу вэ?

READY төлөвтэй процесс CPU авахад бэлэн боловч одоогоор ажиллаагүй байна. RUNNING төлөвтэй процесс яг одоо CPU дээр ажиллаж байна.

4. WAITING төлөв хэзээ үүсдэг вэ?

Процесс I/O, хугацаа, эсвэл тодорхой үйл явдал хүлээх үед WAITING төлөв үүснэ. Манай программд Process 2 эхний матриц нэмсний дараа WAITING болж байгаа.

5. Priority scheduling ямар зарчмаар ажилладаг вэ?

READY процессуудаас хамгийн өндөр priority-той процессыг сонгож ажиллуулна. Энэ программд бага тоо нь өндөр priority гэсэн дүрэмтэй.

6. Context switching гэж юу вэ?

CPU нэг процессоос өөр процесс руу шилжих үйлдлийг context switching гэнэ. Simulation-д үүнийг console дээр хэвлэж харуулсан.

7. Context switching яагаад overhead үүсгэдэг вэ?

Учир нь CPU үндсэн ажлаас гадна өмнөх процессын context хадгалах, дараагийн процессын context сэргээх нэмэлт ажил хийдэг.

8. Энэ лабораторийн simulation бодит үйлдлийн системээс юугаараа ялгаатай вэ?

Энэ нь бодит kernel scheduler биш. Interrupt, CPU register, memory management хийхгүй. Зөвхөн ойлголтыг энгийн C кодоор загварчилсан.

7. ДҮГНЭЛТ

Энэ лабораторийн ажлаар процессын PCB мэдээллийг хадгалах, төлөв солих, priority-гаар сонгох болон context switching-ийн үндсэн санааг C хэл дээр туршиж ойлгов. Програмд нэг мөчид зөвхөн нэг процесс RUNNING төлөвтэй байхаар хийсэн бөгөөд WAITING болон

TERMINATED төлөвүүдийг мөн харуулсан. Даалгаврын шаардлагуудыг биелүүлж, make ашиглан хөрвүүлж ажиллуулж шалгасан.

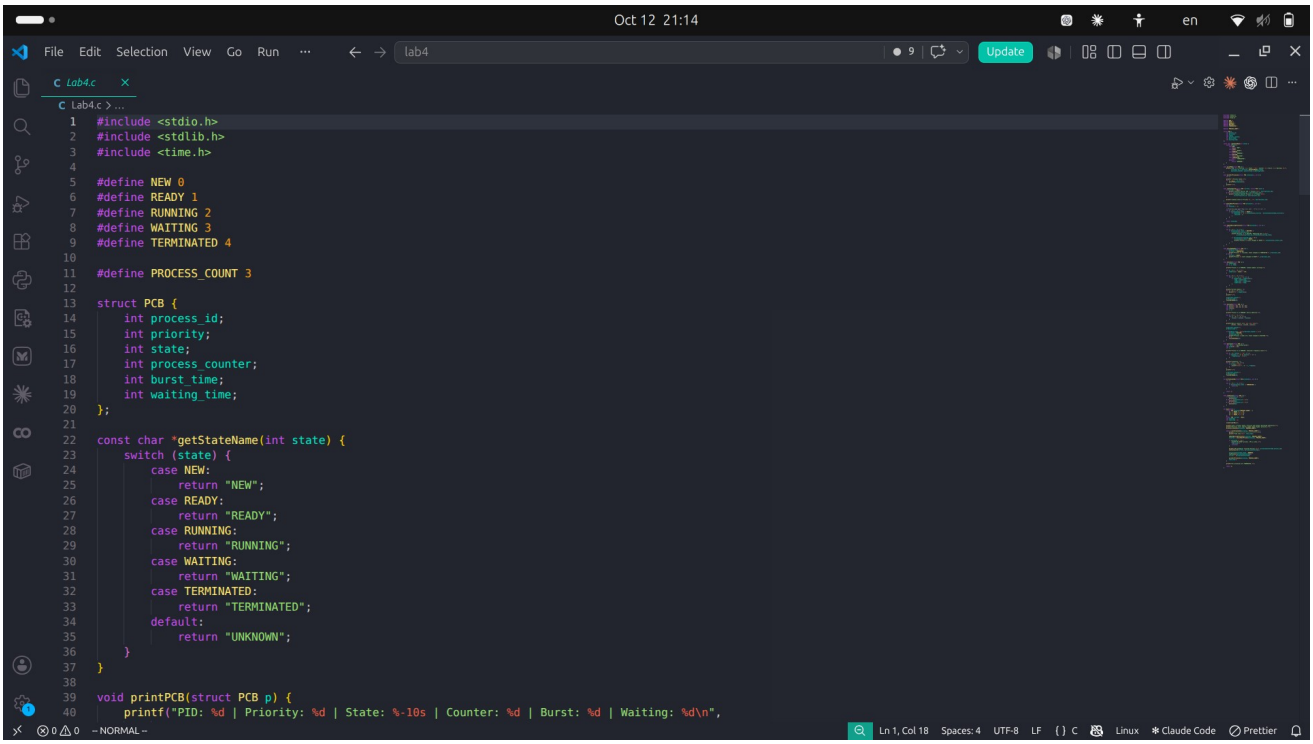
8. ХАВСРАЛТ

Lab4.c файлын эх код:

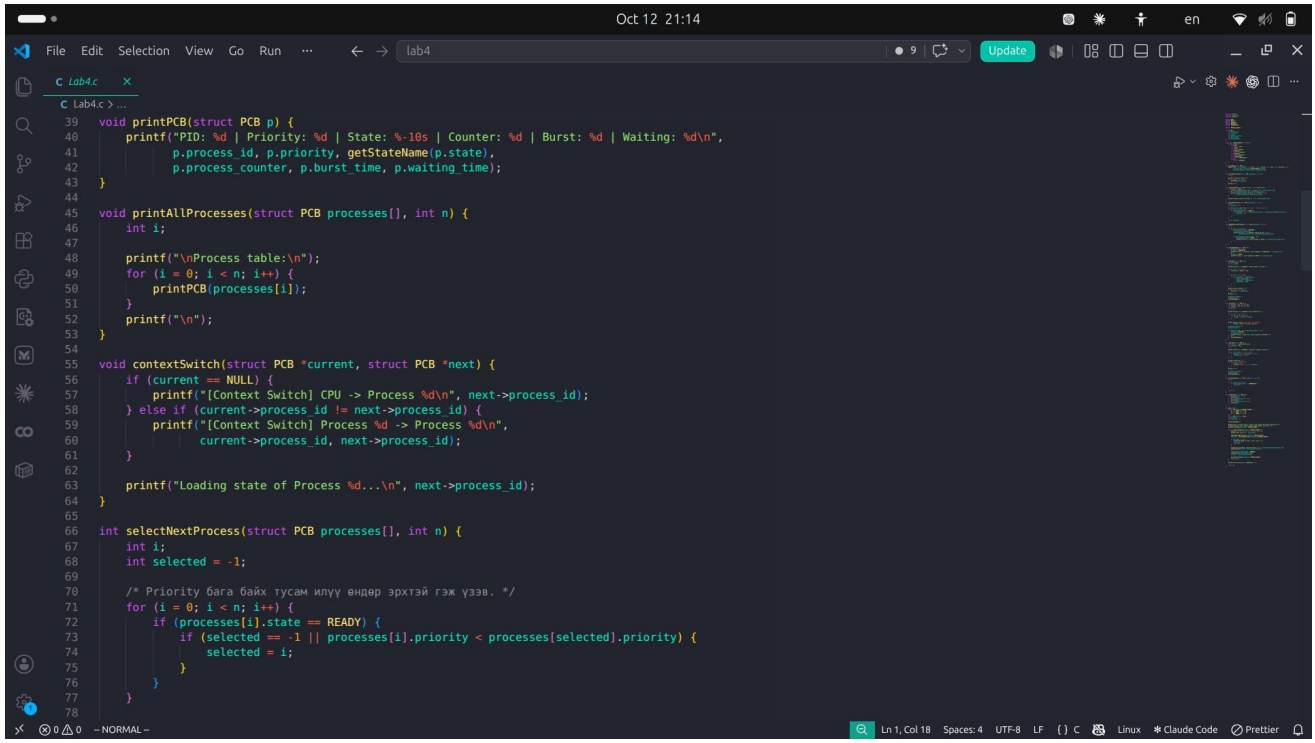
ХАВСРАЛТ Б. ДЭЛГЭЦИЙН ЗУРАГ

Доорх зургууд нь Lab4.c файлын эх код болон scheduler simulation-ийн хэрэгжилтийг ажлын орчин дээр харуулсан дэлгэцийн зургууд юм.

Зураг 1. Lab 4 кодын дэлгэцийн зураг



Зураг 2. Lab 4 кодын дэлгэцийн зураг



```c
void printPCB(struct PCB p) {
    printf("PID: %d | Priority: %d | State: %-10s | Counter: %d | Burst: %d | Waiting: %d\n",
           p.process_id, p.priority, getStateName(p.state),
           p.process_counter, p.burst_time, p.waiting_time);
}

void printAllProcesses(struct PCB processes[], int n) {
    int i;

    printf("\nProcess table:\n");
    for (i = 0; i < n; i++) {
        printPCB(processes[i]);
    }
    printf("\n");
}

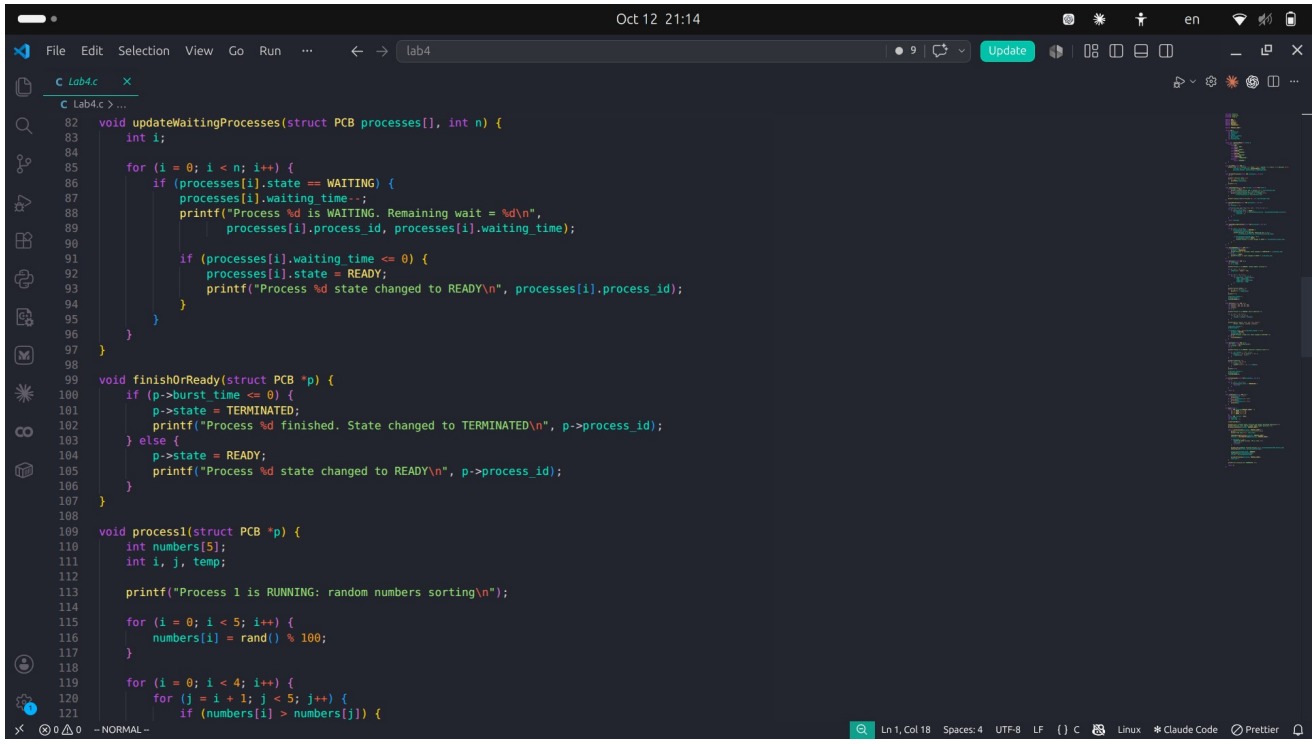
void contextSwitch(struct PCB *current, struct PCB *next) {
    if (current == NULL) {
        printf("[Context Switch] CPU -> Process %d\n", next->process_id);
    } else if (current->process_id != next->process_id) {
        printf("[Context Switch] Process %d -> Process %d\n",
               current->process_id, next->process_id);
    }

    printf("Loading state of Process %d...\n", next->process_id);
}

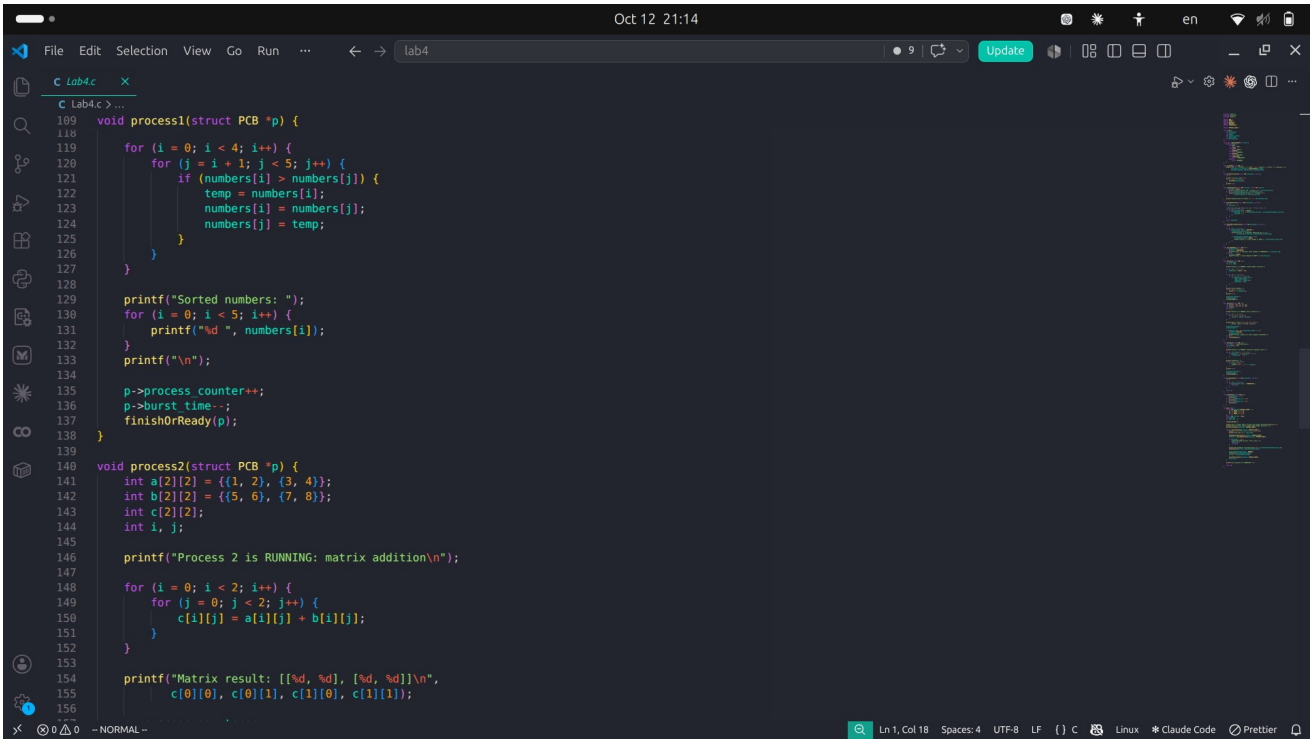
int selectNextProcess(struct PCB processes[], int n) {
    int i;
    int selected = -1;

    /* Priority бага байх тусам илүү өндөр эрхтэй гэж үзэв. */
    for (i = 0; i < n; i++) {
        if (processes[i].state == READY) {
            if (selected == -1 || processes[i].priority < processes[selected].priority) {
                selected = i;
            }
        }
    }
```

Зураг 3. Lab 4 кодын дэлгэцийн зураг



Зураг 4. Lab 4 кодын дэлгэцийн зураг



```c
void process1(struct PCB *p) {

    for (i = 0; i < 4; i++) {
        for (j = i + 1; j < 5; j++) {
            if (numbers[i] > numbers[j]) {
                temp = numbers[i];
                numbers[i] = numbers[j];
                numbers[j] = temp;
            }
        }
    }

    printf("Sorted numbers: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    p->process_counter++;
    p->burst_time--;
    finishOrReady(p);
}

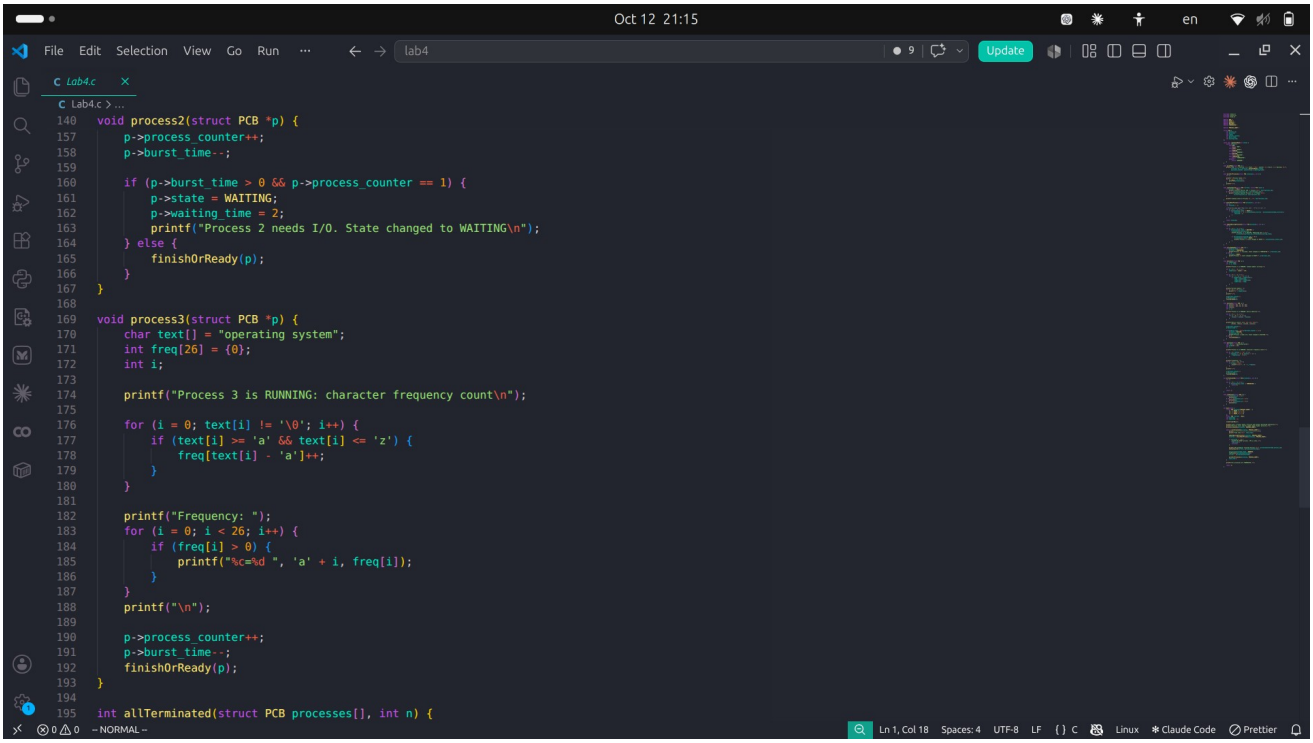
void process2(struct PCB *p) {
    int a[2][2] = {{1, 2}, {3, 4}};
    int b[2][2] = {{5, 6}, {7, 8}};
    int c[2][2];
    int i, j;

    printf("Process 2 is RUNNING: matrix addition\n");

    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++) {
            c[i][j] = a[i][j] + b[i][j];
        }
    }

    printf("Matrix result: [[%d, %d], [%d, %d]]\n",
           c[0][0], c[0][1], c[1][0], c[1][1]);
```

Зураг 5. Lab 4 кодын дэлгэцийн зураг



```c
void process2(struct PCB *p) {
    p->process_counter++;
    p->burst_time--;

    if (p->burst_time > 0 && p->process_counter == 1) {
        p->state = WAITING;
        p->waiting_time = 2;
        printf("Process 2 needs I/O. State changed to WAITING\n");
    } else {
        finishOrReady(p);
    }
}

void process3(struct PCB *p) {
    char text[] = "operating system";
    int freq[26] = {0};
    int i;

    printf("Process 3 is RUNNING: character frequency count\n");

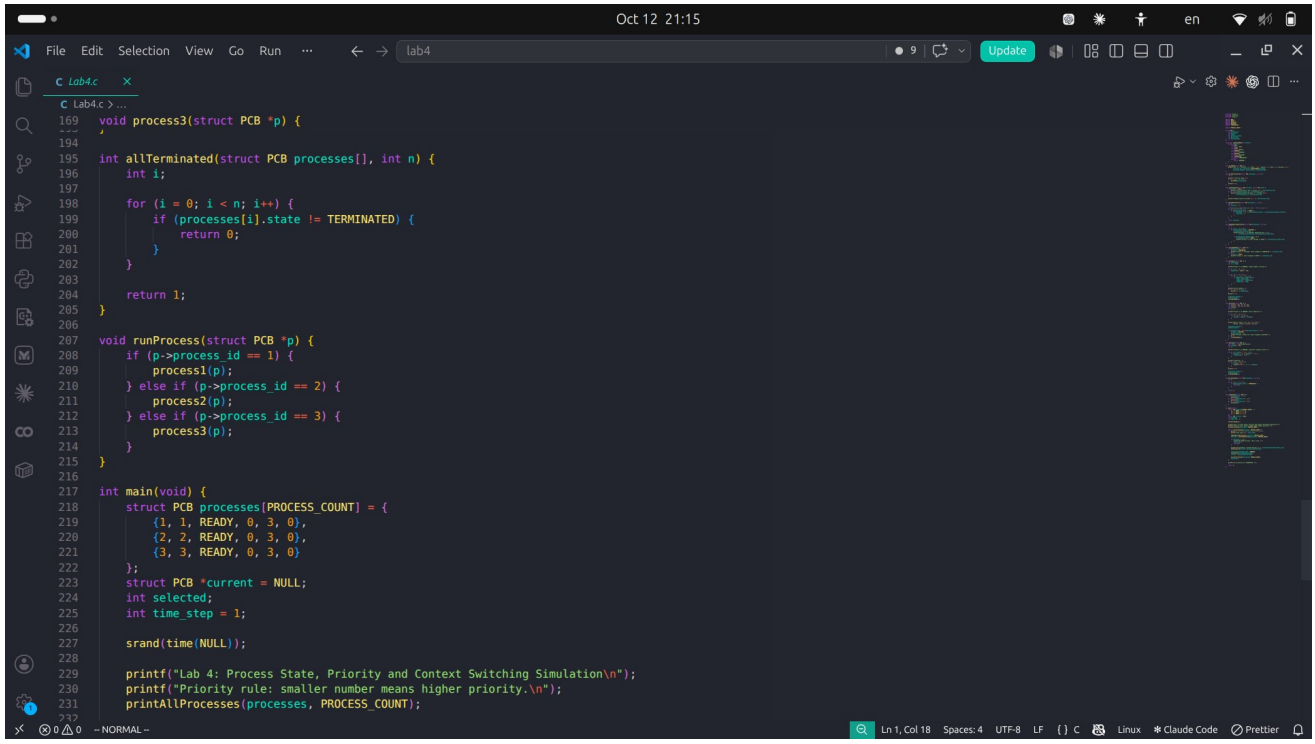
    for (i = 0; text[i] != '\0'; i++) {
        if (text[i] >= 'a' && text[i] <= 'z') {
            freq[text[i] - 'a']++;
        }
    }

    printf("Frequency: ");
    for (i = 0; i < 26; i++) {
        if (freq[i] > 0) {
            printf("%c=%d ", 'a' + i, freq[i]);
        }
    }
    printf("\n");

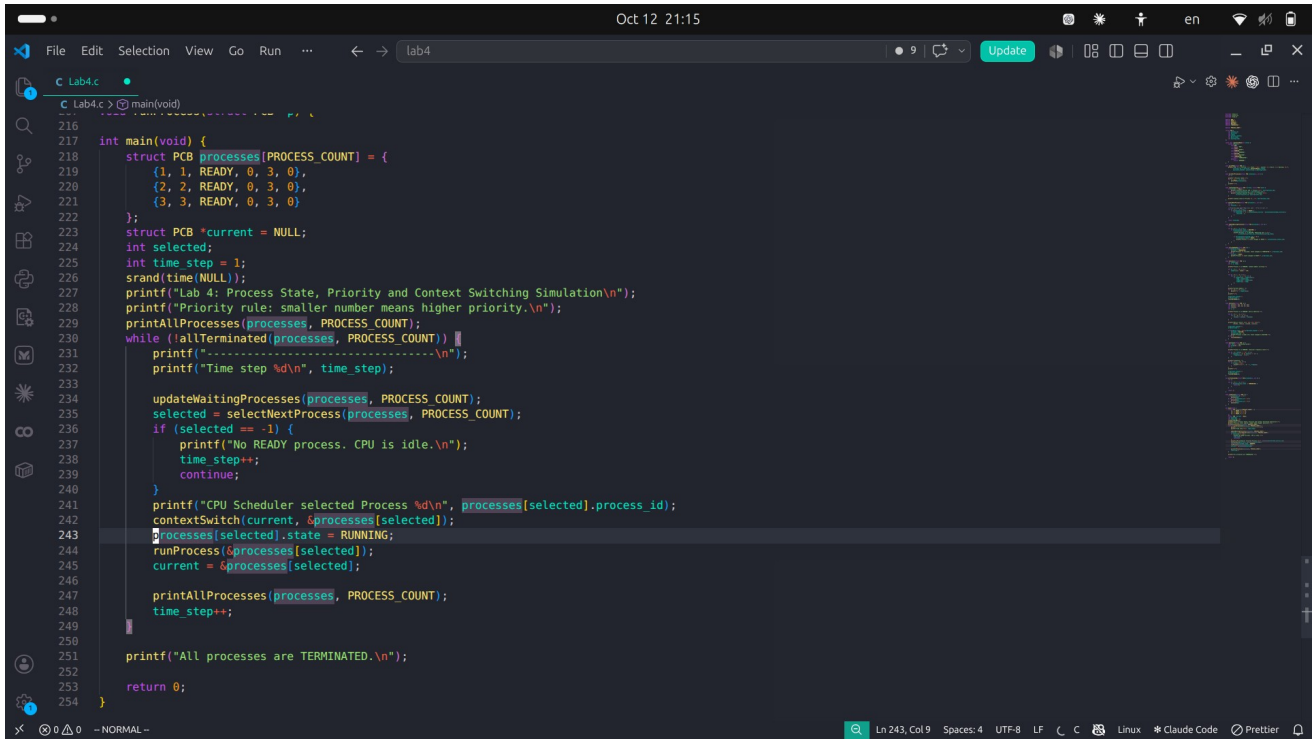
    p->process_counter++;
    p->burst_time--;
    finishOrReady(p);
}

int allTerminated(struct PCB processes[], int n) {
```

Зураг 6. Lab 4 кодын дэлгэцийн зураг



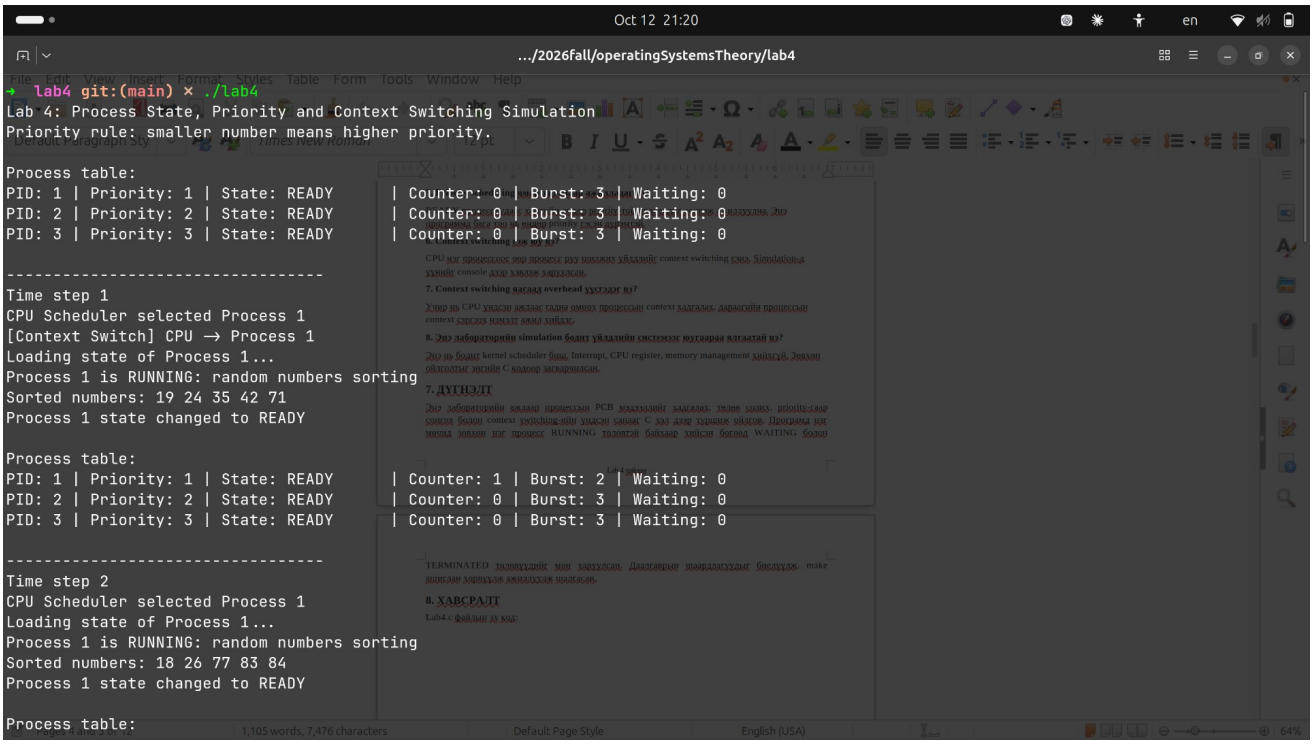
Зураг 7. Lab 4 кодын дэлгэцийн зураг



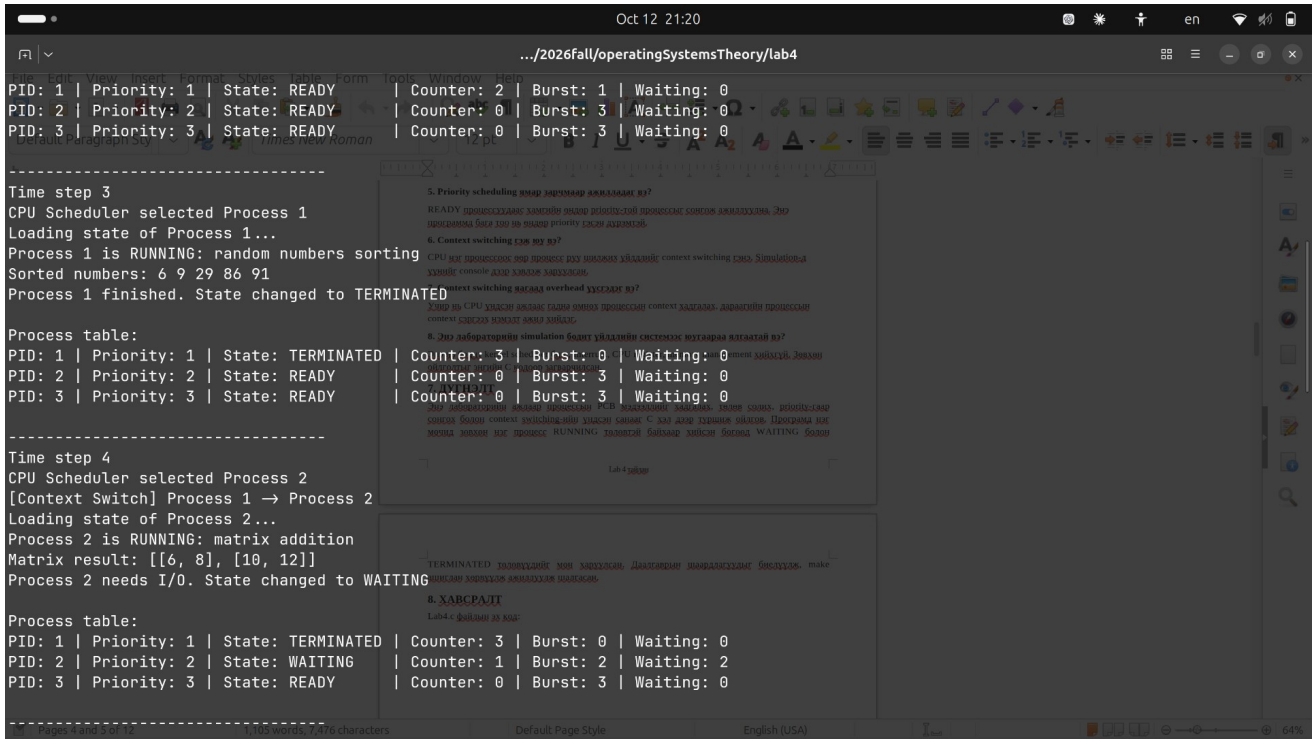
ХАВСРАЛТ В. ПРОГРАМЫН ГАРАЛТЫН ДЭЛГЭЦИЙН ЗУРАГ

Доорх зургууд нь Lab4 программыг ажиллуулах үед гарсан scheduler simulation-ийн гаралтыг дарааллаар нь харуулсан.

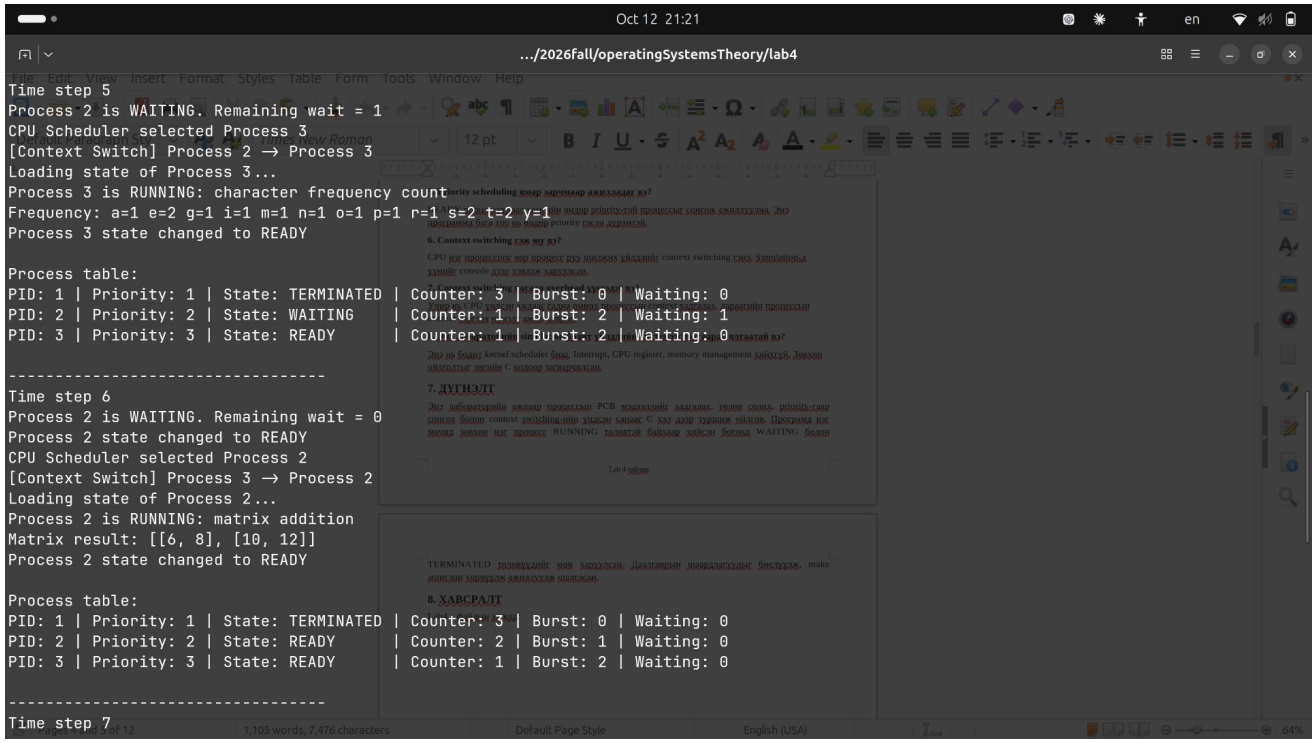
Зураг 8. Lab 4 програмын гаралтын дэлгэцийн зураг



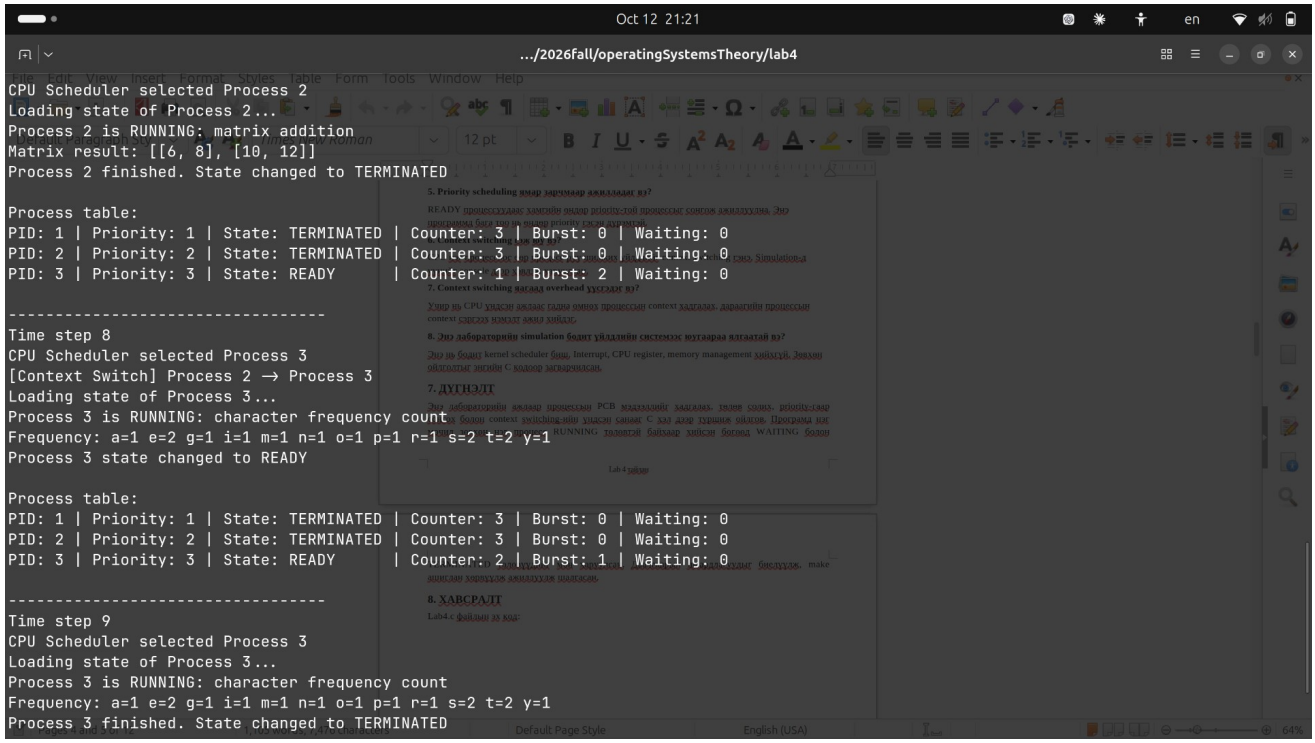
Зураг 9. Lab 4 програмын гаралтын дэлгэцийн зураг



Зураг 10. Lab 4 програмын гаралтын дэлгэцийн зураг



Зураг 11. Lab 4 програмын гаралтын дэлгэцийн зураг



Зураг 12. Lab 4 програмын гаралтын дэлгэцийн зураг

